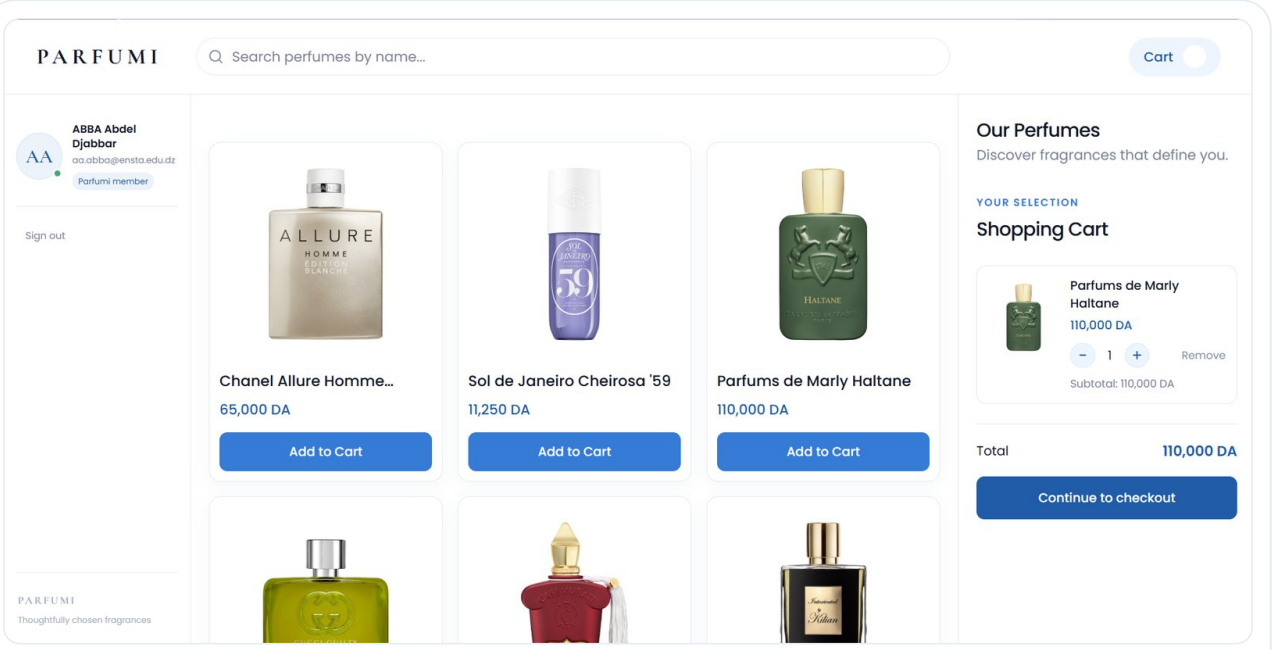
TD-TP 3

Parfumi Store

TD-TP 3 | Web and Mobile Development

STUDENT	Abdeldjabbar
PROJECT TYPE	Full-stack perfume e-commerce application
FRONTEND	React, Vite, Redux Toolkit
BACKEND	Node.js, Express, MongoDB, Mongoose
EVIDENCE	30 supplied screenshots + source-code snapshots

A practical record of what I built, connected, tested, and learned while completing this project.



STOREFRONT

The storefront: product catalogue, search, and shopping cart.

This report documents TD-TP 3 only. It does not include or describe TD-TP 2 as part of this submission.

01

Project overview

Parfumi Store is a perfume storefront connected to a backend API and a MongoDB database. I worked on keeping the shopping experience connected to real product data while adding account access and administrator-only product management.

The project combines a React frontend, Redux state for the cart, an Express API, and MongoDB collections for users and products.

Main features

01
Browse perfume products and search by product name.

02
Add products to the cart, change quantities, remove items, and calculate the total.

03
Register and sign in with email and password.

04
Allow administrator users to create, update, and delete products.

05
Protect API routes with token verification and role checks.

06
Limit repeated requests to the API and login endpoint.

How the parts connect

FRONTEND
React components render the store and call the API through `src/services/api.js`.

CART STATE
Redux Toolkit stores cart items and quantity changes in client-side state.

BACKEND
Express defines authentication and product routes, middleware, and rate limits.

DATABASE
Mongoose models describe users and products stored in MongoDB.

> The cart is currently frontend-only. It is not saved as an order in MongoDB, and checkout is not implemented in this version.

SCOPE NOTE
This package is organized as a TD-TP 3 submission. The application source is kept, while dependencies, local secrets, and build output are excluded from the hand-in archive.

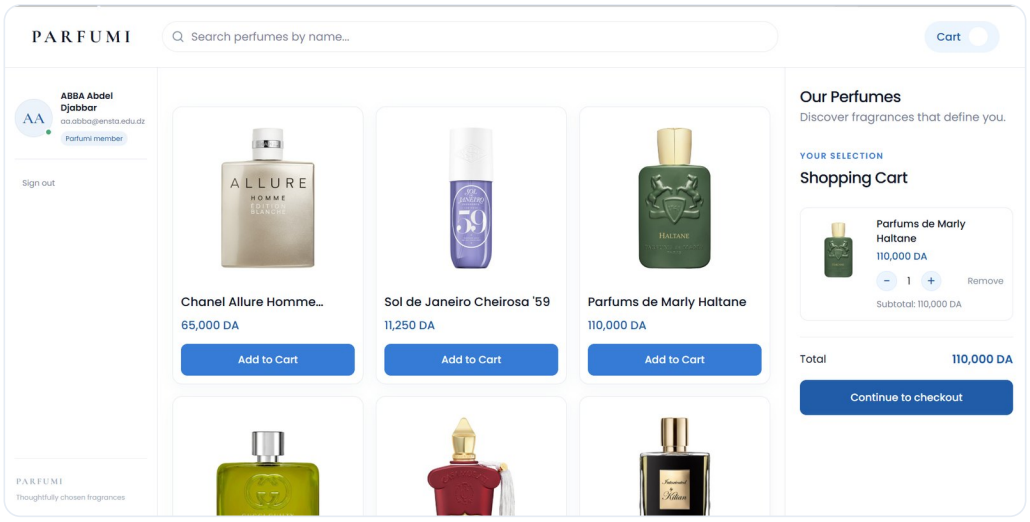
02

Frontend experience

The frontend keeps the shopping flow simple: the product catalogue is visible in the main area, search is available in the header, and the cart stays beside the products. The account sidebar changes according to the signed-in user.

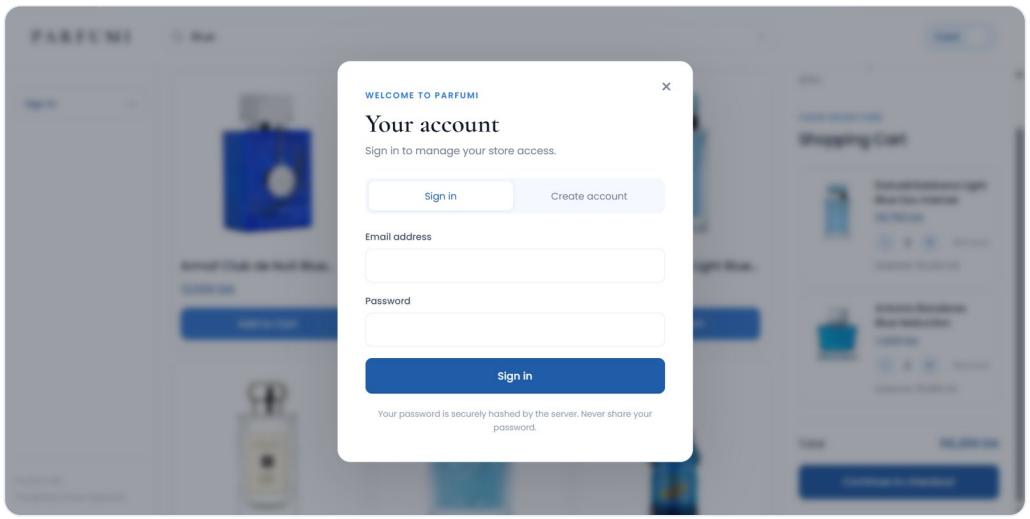
The product list is loaded from `GET /api/products`. Search filters the loaded product names. The cart uses Redux Toolkit actions to add items, increase or decrease quantities, remove items, and calculate totals.

Storefront and account access



EVIDENCE 01

Storefront catalogue and cart layout.



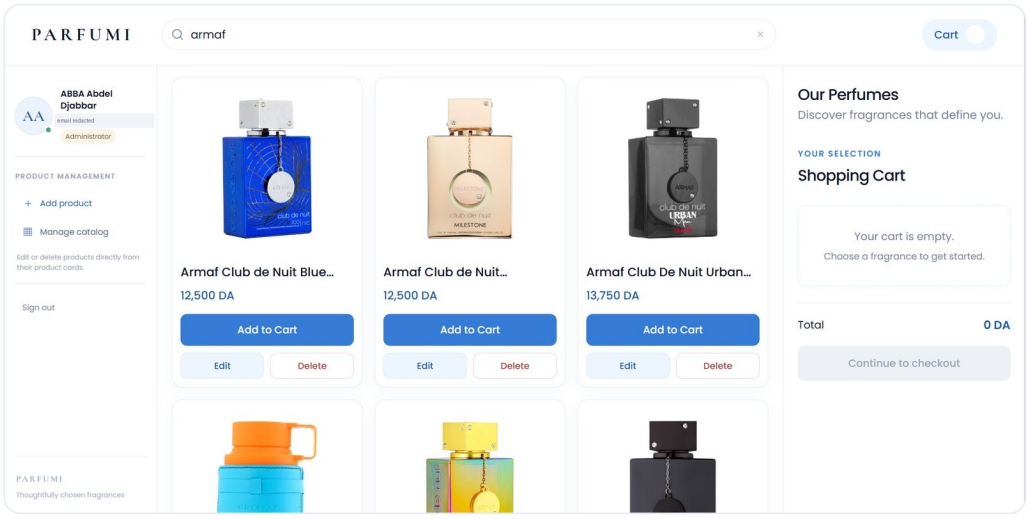
EVIDENCE 05

Sign-in modal for local email/password authentication.

03

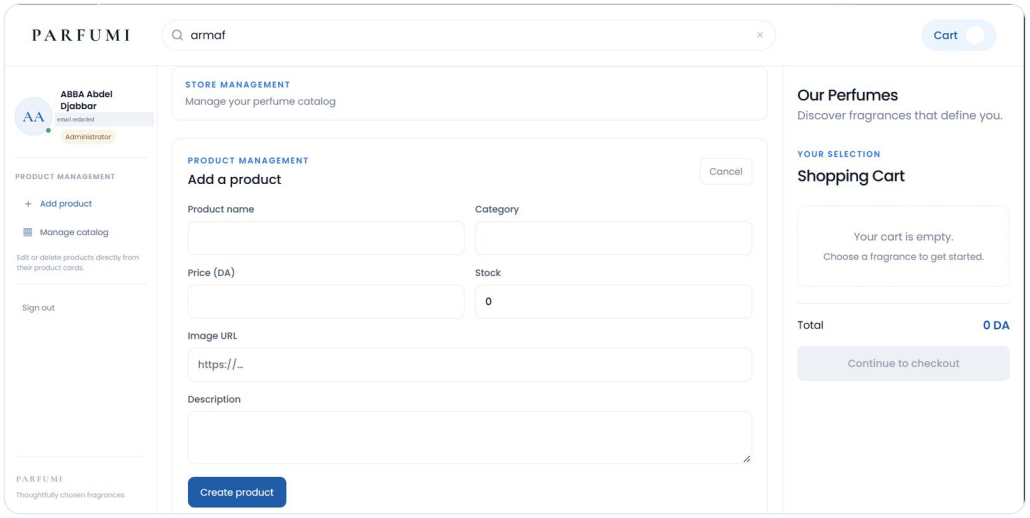
Administrator product management

After signing in with an administrator account, the sidebar shows product-management actions. The catalogue cards also display Edit and Delete controls. The product form collects the fields expected by the backend model.



EVIDENCE 18

Administrator catalogue view. Account email is redacted in this copy.



EVIDENCE 19

Add-product form. Account email is redacted in this copy.

The frontend sends create, update, and delete requests with the current session token. The backend still checks the administrator role; hiding buttons in the interface is not the only protection.

04

Backend and database

The backend is an Express application. It connects to MongoDB through Mongoose and exposes endpoints for authentication and product operations. The database has two main models: `User` and `Product`.

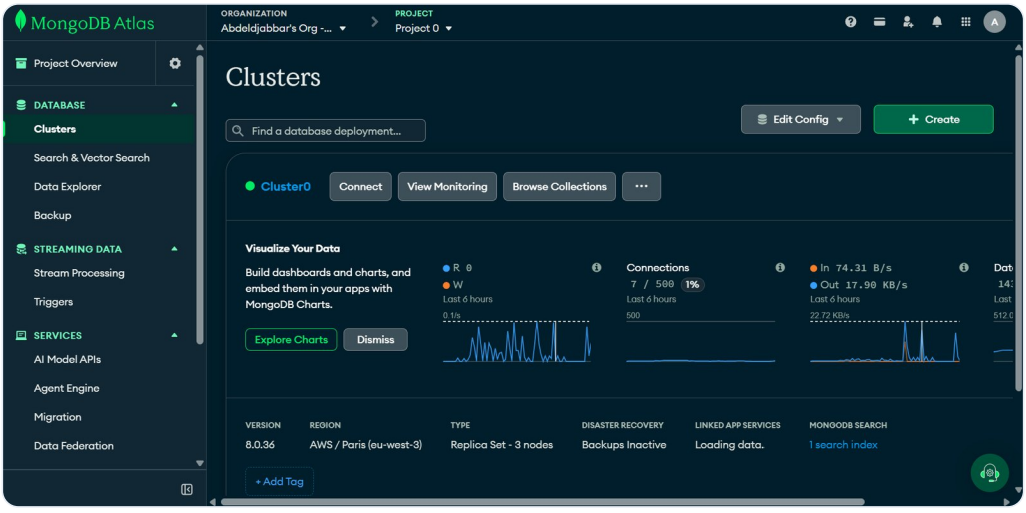
Data models

MODEL	FIELDS USED IN THE PROJECT
User	`name email passwordHash role timestamps`
Product	`name description price category imageUrl stock timestamps`

API routes

METHOD	ENDPOINT	ACCESS
POST	`/api/auth/register`	Public
POST	`/api/auth/login`	Public, rate limited
GET	`/api/products` and `/api/products/:id`	Public
POST / PUT / DELETE	`/api/products` and `/api/products/:id`	Admin only

Database configuration evidence



EVIDENCE 09

MongoDB Atlas cluster overview.

The screenshot folders also include the users collection and the administrator role used during product-management testing.

05

Authentication and authorization

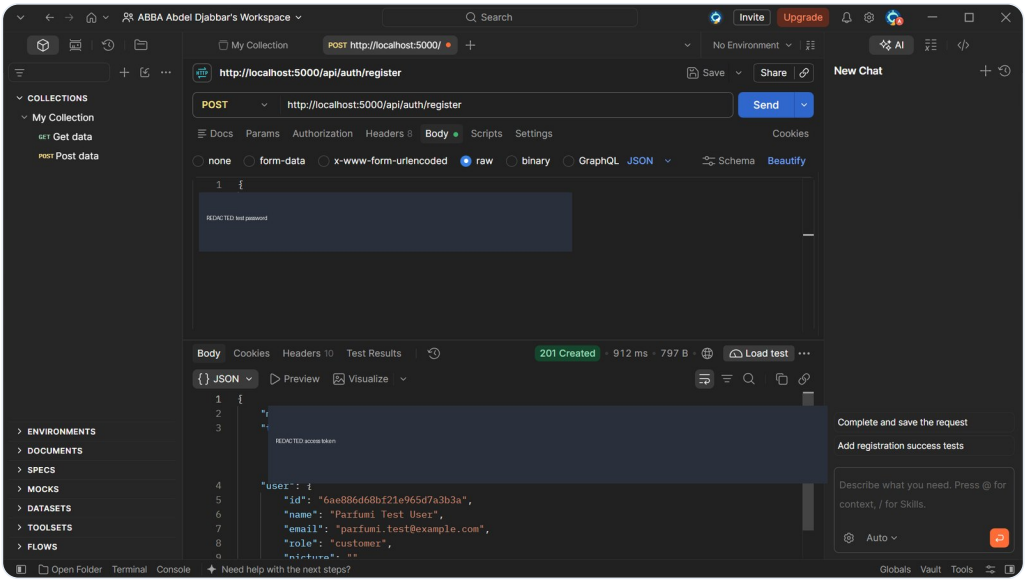
Local registration checks the required fields and validates the email and password lengths. The backend hashes the password with bcryptjs before storing it. During login, the server compares the submitted password with the stored hash and returns a signed JWT when the credentials are valid.

The local JWT uses HS256 and expires after one day. Protected requests send it in the `Authorization: Bearer` header. The authentication middleware verifies the signature and expiry before allowing the request to continue.

Administrator access

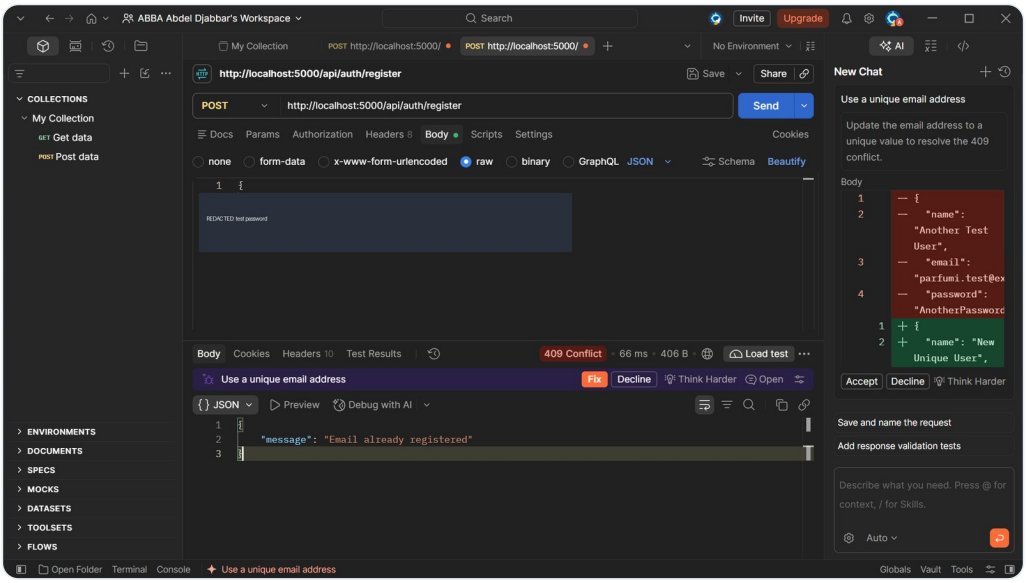
Product creation, update, and deletion use both the authentication middleware and `requireAdmin`. A customer token alone is not enough. Newly registered local accounts are assigned the `customer` role. For admin testing, the account role was changed in the MongoDB users collection to `admin`, then a fresh login token was used.

Registration evidence



EVIDENCE 12

Successful registration (201 Created). Password and access token are redacted.



EVIDENCE 13

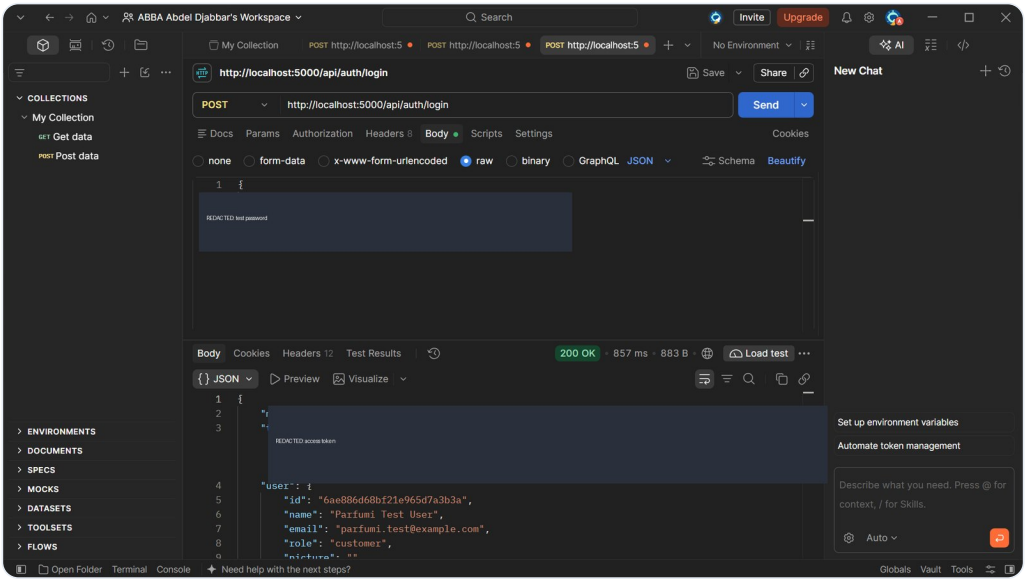
Duplicate email rejected (409 Conflict). Password is redacted.

06

API testing

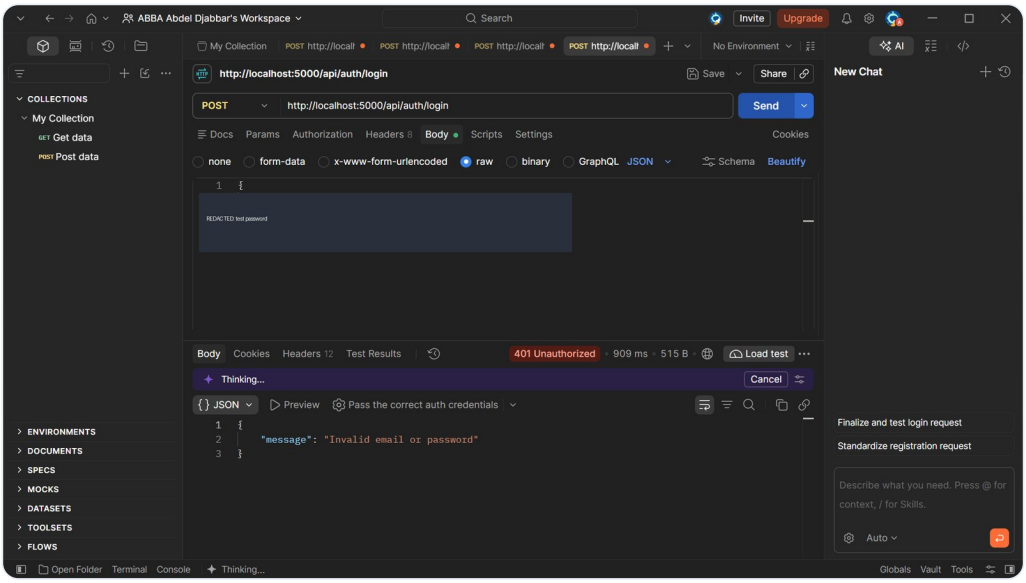
I used Postman to send requests to the local API. The supplied screenshots record both successful operations and cases where the API correctly rejected a request. The Postman collection is included with its saved token values cleared.

Login checks



EVIDENCE 14

Successful login (200 OK). Password and token are redacted.



EVIDENCE 15

Incorrect login credentials rejected (401 Unauthorized).

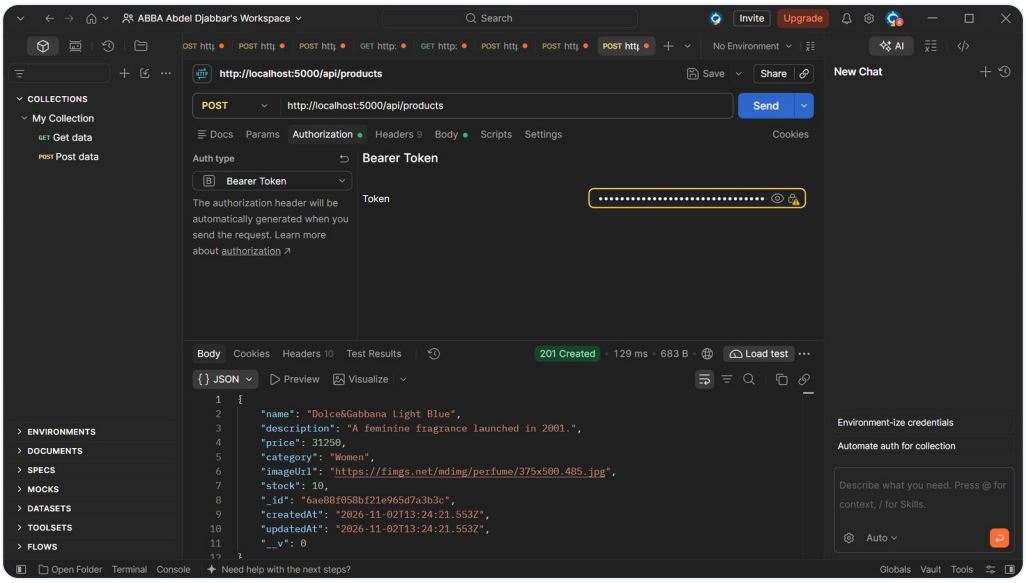
The evidence set also includes public product retrieval and tests for protected requests.

07

Product operations

The API supports retrieving the catalogue and an individual product. Administrator requests can create, update, and delete products. The Mongoose schema defines the required product fields, and invalid identifiers or invalid data are handled with error responses.

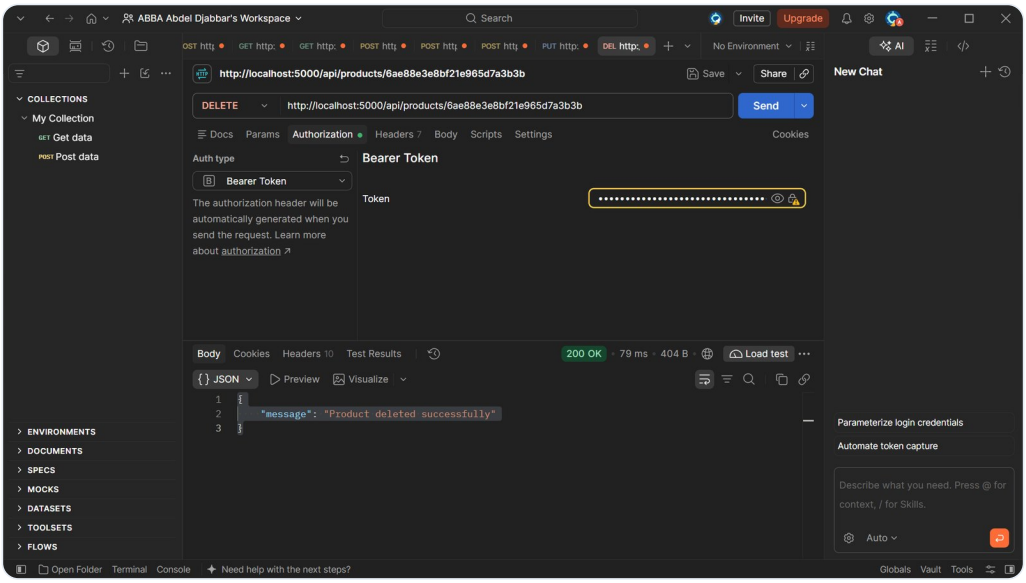
Product creation



EVIDENCE 22

Product creation returned 201 Created. Bearer token is masked in Postman.

Product deletion



EVIDENCE 24

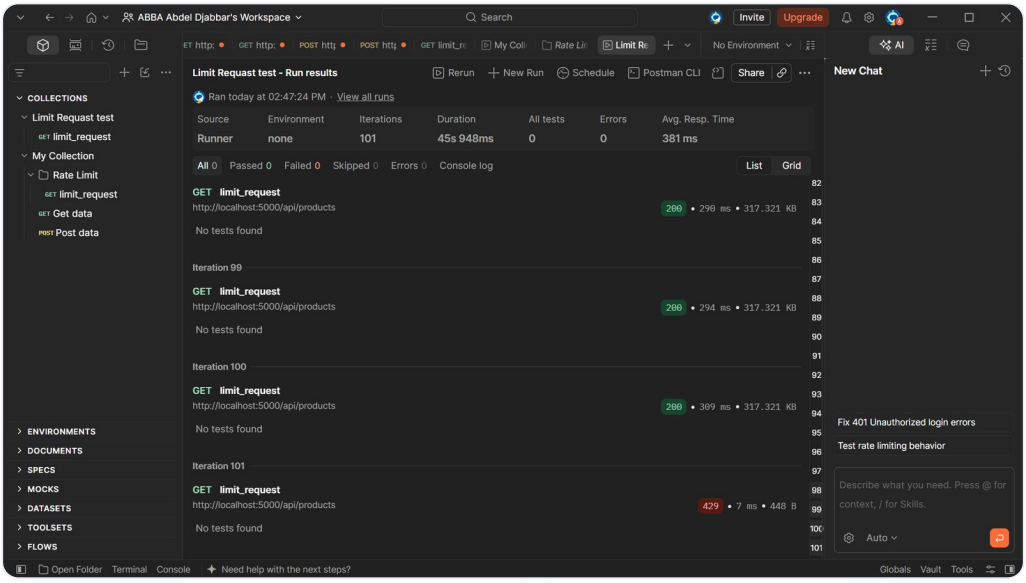
Product deletion returned 200 OK. Bearer token is masked in Postman.

08

Validation and rate limiting

The code includes checks for missing and invalid tokens, administrator authorization on product write routes, invalid product identifiers, and product-field validation. The API has two configured rate limits: 100 requests per 15 minutes for the general `/api` routes and 5 attempts per 15 minutes for login.

General API rate-limit evidence



EVIDENCE 30

Postman Collection Runner: 101 iterations. The final request is shown with HTTP 429.

Evidence status

AREA	STATUS	EVIDENCE
Registration / duplicate registration	Verified	201 Created and 409 Conflict shown in screenshots.
Login / invalid credentials	Verified	200 OK and 401 Unauthorized shown.
General API rate limit	Verified	101 iterations shown; final request returned 429.
Login rate limit	Partial	Limiter is implemented in code (5 attempts / 15 minutes), but the supplied screenshots do not clearly prove the final 429 result.
Auth0 OAuth2 protected endpoint	Not completed	Auth0-related frontend/backend code exists, but the final protected-endpoint Postman verification was not completed. This report does not claim it passed.

09

What I learned and what remains

This project helped me understand how the frontend and backend work together. I moved from displaying products in a page to loading them from an API, storing users in a database, protecting routes, and testing real HTTP responses.

I also learned that an error response is useful evidence. A duplicate registration should not create another account, a wrong password should not sign in, and a customer should not be allowed to manage the product catalogue. Seeing these responses in Postman helped me connect the code to the behavior of the application.

Current limitations

- The shopping cart is held in Redux client state and is not persisted in MongoDB.
- The checkout button is present in the interface, but order creation and payment are outside the current implementation.
- Auth0 helper and callback code exists, but the visible sign-in dialog exposes local email/password sign-in only. The final Auth0 protected-request Postman test was not completed.
- The login-specific 429 outcome is not established by the supplied screenshots, even though the rate limiter is configured in the code.
- The archive used for packaging included a local backend `.env` file. It was excluded, and placeholders are provided instead.

A practical next step

Before submitting, create a local `backend/.env` from the example file, install dependencies with `npm install` in the root and backend folders, run the backend and frontend locally, and check the application in the browser. Rerun the login rate-limit test if it is required by the assignment.

MY REFLECTION

I did not understand every part immediately. I had to test requests, read responses, correct mistakes, and connect what I saw in Postman and MongoDB with the code I was writing. This process made the project more meaningful to me: I was not only adding features, I was learning how the pieces behave together.

The code snapshots are copies of selected source files from the supplied project. The screenshot folders contain all 30 supplied images; credential-bearing parts have been redacted in the submission copies. The packaging pass checked backend JavaScript syntax. A frontend production build and ESLint run were not performed because dependencies were intentionally excluded.